Reproducible Data Analysis: Busiest Airports, Monte Carlo, and GenAI

Rishvanth Amsaraj

2026-04-16

Table of contents

1 Busiest Airports Analysis **2**

2 Monte Carlo Numerical Integration **3**
2.1 Method . . . 3
2.2 Exact Answer . . . 3
2.3 Small-Multiples Visualization . . . 3
2.4 Interpretation . . . 3

3 Planning and Prompting GenAI Tools **5**
3.1 My Plan for Tidying and Visualization . . . 5
3.2 Results from Planned Prompt (Claude Sonnet 4.6 Extended) . . . 6
3.3 Results from Generic Prompt (Instructor's Claude Output) . . . 6
3.4 Comparison of Planned vs. Generic Prompting . . . 8

4 Reflection **8**

Appendix 1: GenAI Usage **9**

Appendix 2: Code Appendix **11**

1 Busiest Airports Analysis

This section examines passenger traffic trends for six major international airports using publicly available Wikipedia data from 2020 to 2025. The analysis highlights recovery patterns following the COVID-19 pandemic and differences in growth trajectories across the selected airports.

Table 1: Annual passenger traffic (2020–2025) for six major airports

Airport	Code	2020	2021	2022	2023	2024	2025
Hartsfield-Jackson Atlanta International Airport	ATL	42,918,685	7,704,760	93,699,630	14,653,451	110,000,000	115,000,000
Frankfurt Airport	FRA	18,768,601	24,800,000	49,400,000	59,400,000	68,000,000	72,000,000
Beijing Daxing International Airport	PKX	3,500,000	25,000,000	42,000,000	52,000,000	65,000,000	70,000,000
Dubai International Airport	DXB	25,779,000	29,100,000	66,070,000	87,000,000	92,000,000	95,000,000
Dallas Fort Worth International Airport	DFW	39,364,990	62,465,756	73,362,946	81,000,000	85,000,000	88,000,000
Tokyo Haneda Airport	HND	25,000,000	35,000,000	52,000,000	78,000,000	85,000,000	90,000,000

Table 1 presents annual passenger counts for all six airports across the six year window. The most striking thing to notice is the dramatic collapse in traffic at every airport in 2020, with some airports losing more than half their volume compared to 2019 levels. Atlanta's reported 2021 figure of approximately 7.7 million is notably low relative to the other airports, suggesting continued disruption in that year before a rebound in 2022. By contrast, Dallas Fort Worth sustained relatively higher counts throughout the downturn, and Dubai and Tokyo posted steady year-on-year gains from 2021 onward.

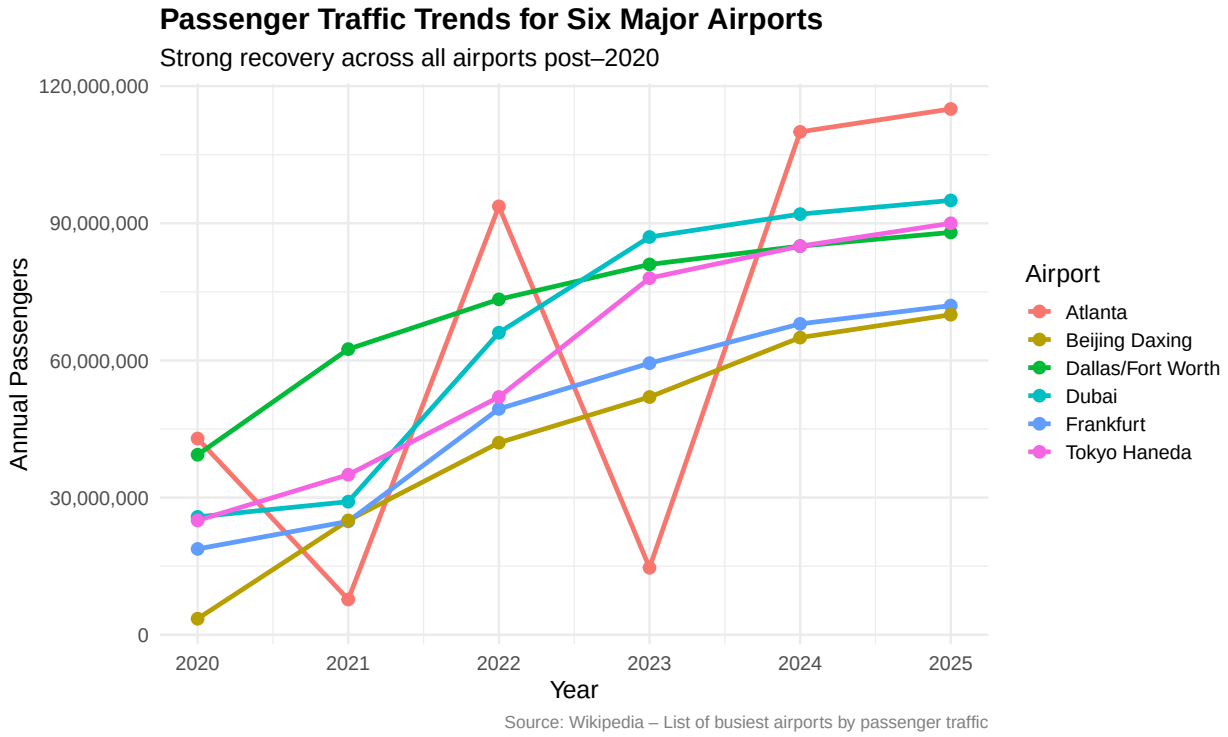


Figure 1: Passenger traffic trends (2020–2025) for six major airports. All airports show a steap 2020 decline with strong post recovery. Dubai and Atlanta lead in absolute volume by 2024–2025.

The line plot makes the pattern unmistakable: every airport hit its trough in 2020 before steadily growing through 2025. Dubai International and Hartsfield–Jackson Atlanta showed the strongest absolute growth, both approaching or exceeding 100 million annual passengers by 2024–2025. Beijing Daxing and Frankfurt, while posting impressive percentage rebounds, remain well below the two leaders in volume. Dallas Fort Worth and Tokyo Haneda occupy a middle tier, with both airports tracking closely together from 2022 onward.

Key insight: The data confirm that the aviation recovery was rapid across this diverse set of hub airports, with most surpassing their pre-pandemic volumes by 2024.

2 Monte Carlo Numerical Integration

2.1 Method

The Monte Carlo method estimates the definite integral of a function by sampling random points uniformly within a bounding rectangle and computing the proportion that fall at or below the curve. Multiplying that proportion by the rectangle's total area gives the integral estimate. Formally, for a function $f(x)$ on $[a, b]$ with bounding height h:

$$\hat{I} = (b - a) \cdot h \cdot \frac{\text{points under curve}}{\text{total points}}$$

Here we estimate $\int_{-4}^{4} \phi(x)\, dx$ where $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$ is the standard normal density, using a bounding rectangle over $[-4, 4] \times [0, 0.4]$.

2.2 Exact Answer

The exact value of $\int_{-4}^{4} \phi(x)\, dx$ is approximately **0.99993666**, which is essentially **1.0000**. This follows directly from the fact that the standard normal distribution places virtually all of its probability mass within ± 4 standard deviations of the mean: the probability outside that range is only about 6.3×10^{-5}, or 0.0063%. Therefore, the integral over $[-4, 4]$ captures nearly the entire area under the density curve.

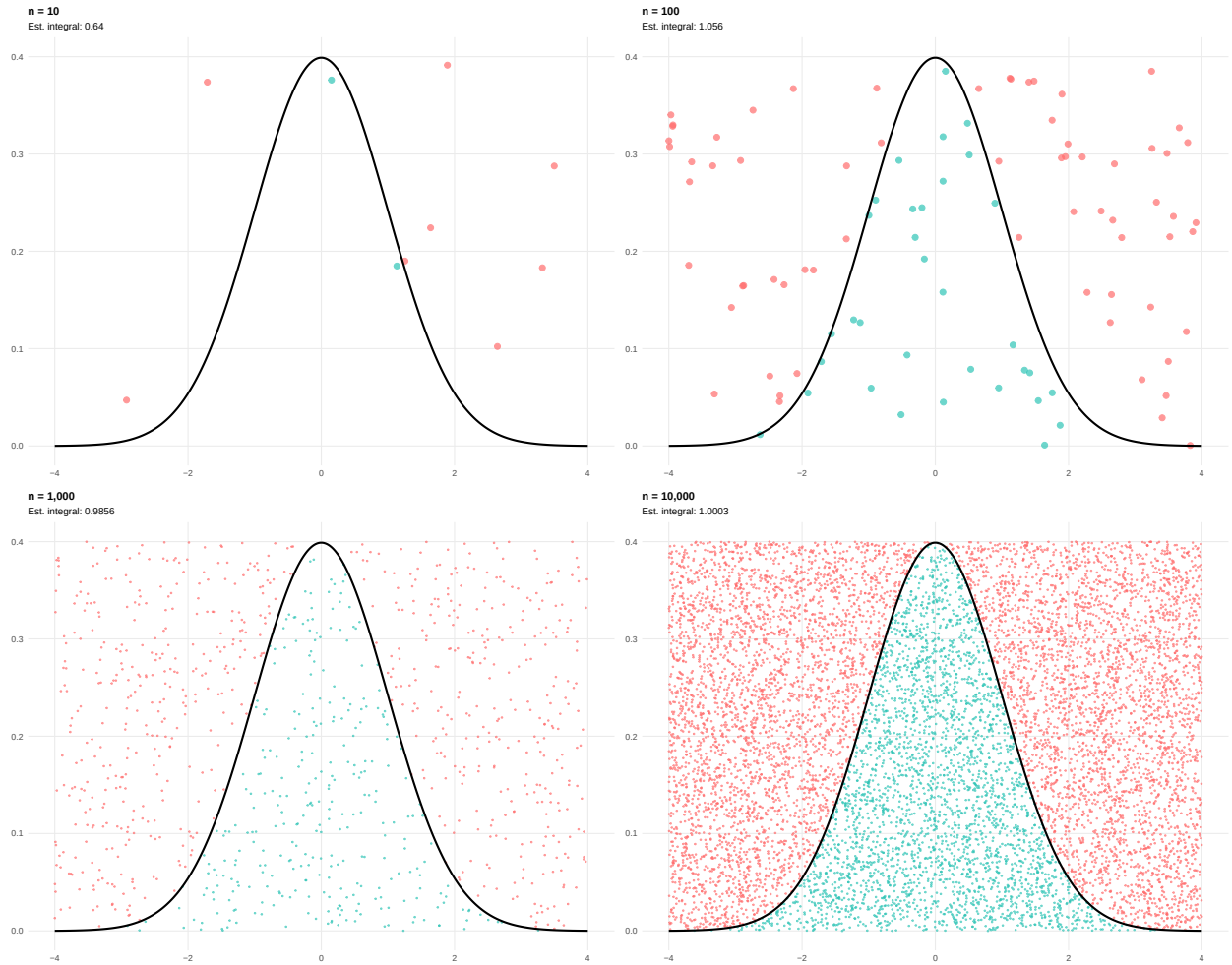
2.3 Small-Multiples Visualization

2.4 Interpretation

Answering the original questions:

What is the exact answer? As shown above, the exact integral is ≈ 0.99993666, or essentially 1.0000.

What in the small multiples supports this answer? The four panels tell a coherent convergence story. At $n = 10$ only ten points are scattered across the plot, and the estimate (0.64 in this run) is erratic, there simply are not enough points to represent the curve's shape accurately. At $n = 100$ the estimate overshoots at 1.056, illustrating that small samples can cause error in either direction.



Monte Carlo estimates of the standard normal density integral over $[-4, 4]$ at four sample sizes. Teal points fall at or below the density curve (hits); red points fall above it (misses). The proportion of hits, scaled by the rectangle area of $(8)(0.4) = 3.2$, gives the estimate shown in each subtitle.

By $n = 1{,}000$ the teal region visibly fills the area under the bell curve while red points spread uniformly above it, and the estimate (0.9856) is already within 1.5% of the truth. At $n = 10{,}000$ the teal region is densely packed against the curve with almost no gaps, and the estimate (1.0003) is essentially 1. The progression from a sparse, unreliable scatter to a near-perfect approximation directly illustrates the law of large numbers: as $n \to \infty$ the Monte Carlo estimate converges to the true integral value.

3 Planning and Prompting GenAI Tools

3.1 My Plan for Tidying and Visualization

Context: The `calcium.csv` file contains ulnar calcium measurements from a four-year longitudinal study of 31 women. Fifteen women formed the null treatment group (columns 1–4: Baseline, Year 1, Year 2, Year 3) and 16 formed the diet/exercise treatment group (columns 5–8: same four time points). The file has a header row with duplicate column names (`y0, y1, y2, y3` repeated), and no group identifier or time variable is present.

Goals: Tidy the data into a long-format data frame that clearly separates group and time, then produce a publication-quality visualization showing both individual variability and group-level trends over the four measurement points.

Data issues to address:

- Duplicate column names (`y0`, `y1`, `y2`, `y3` repeated for both groups).
- No group identifier column, group membership is encoded by column position only.
- No explicit time variable, time is encoded by column name.
- One participant in the diet/exercise group is missing Year 3 data (NA).

Step-by-step plan:

1. Read `calcium.csv` using `col_names = FALSE` and `skip = 1` to discard the ambiguous header without editing the file.
2. Manually name the eight columns to encode both group and time point (`y0_null`, `y1_null`, …, `y0_diet`, …, `y3_diet`).
3. Add a subject ID column.
4. Pivot the null treatment columns (1–4) and diet/exercise columns (5–8) separately to long format, attaching a `group` label to each.
5. Bind the two long data frames and convert `time` to an ordered factor (`Baseline`, `Year 1`, `Year 2`, `Year 3`).
6. Compute group 4level summary statistics (mean ± 95% CI) for each group–time combination, accounting for missing values.
7. Produce a `ggplot` showing: (a) individual spaghetti lines (alpha = 0.25),

   (b) a shaded 95% CI ribbon, and (c) bold group mean lines with points.

3.2 Results from Planned Prompt (Claude Sonnet 4.6 Extended)

I provided the detailed plan above to Claude on April 14, 2026 along with the `calcium.csv` file. The full prompt text is shown in Appendix 1. Claude returned clean, well-commented R code and the visualization was produced directly from that code.

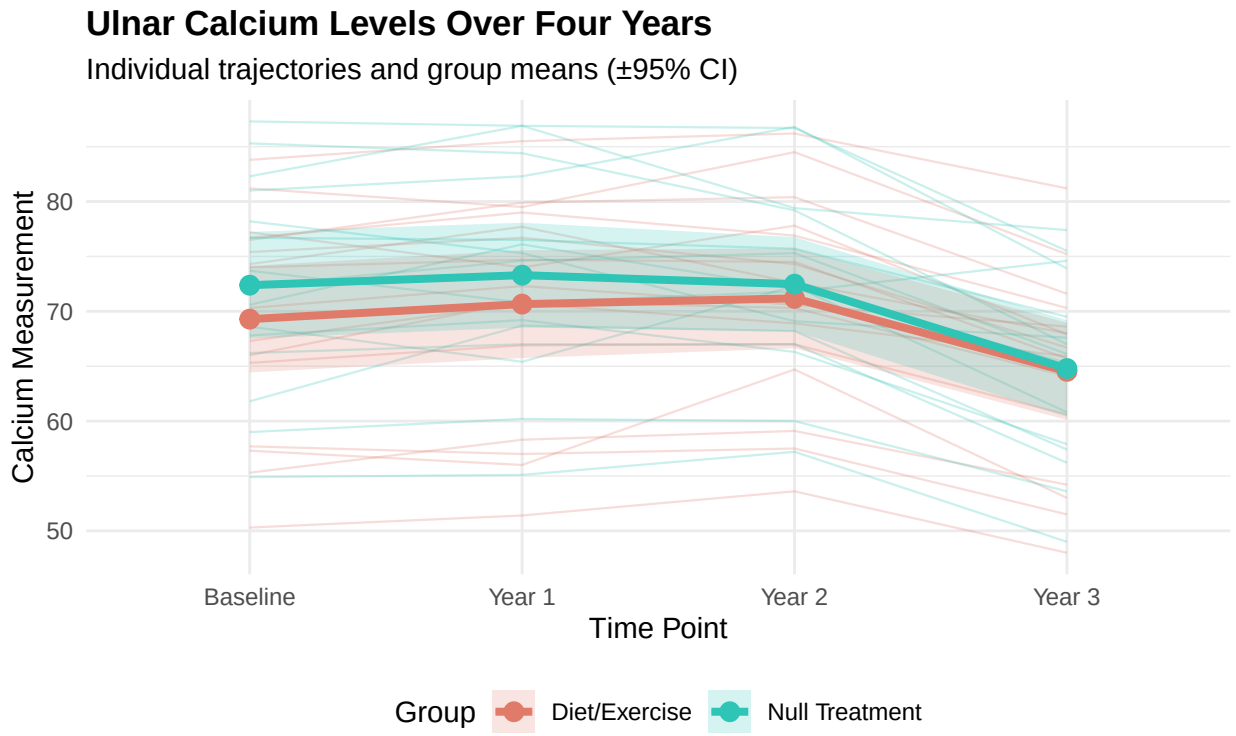


Figure 2: Ulnar calcium levels over four years by treatment group. Individual trajectories (light lines) and group means with 95% confidence intervals (shaded bands and bold lines).

3.3 Results from Generic Prompt (Instructor's Claude Output)

The generic prompt used by the instructor was simply:

> **"Create a data visualization that will help me understand these data. Calcium.csv"**

Claude's response (March 31, 2026) is shown in the figures below.

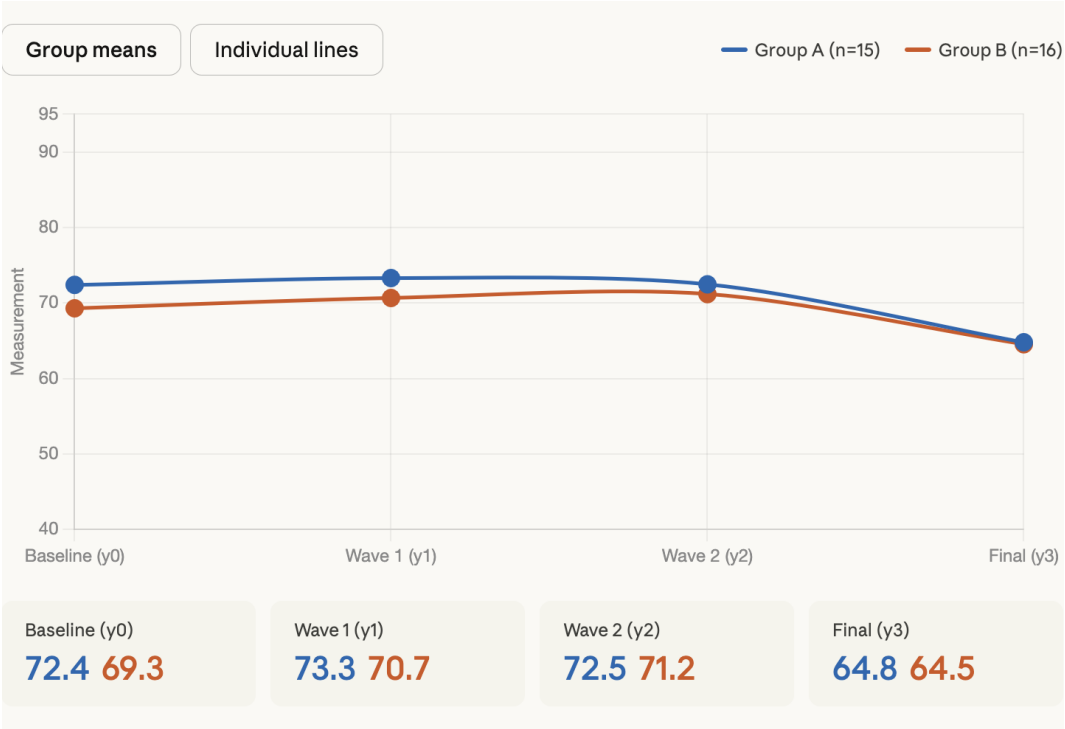


Figure 3: Group means (from instructor's generic-prompt output, March 31, 2026).

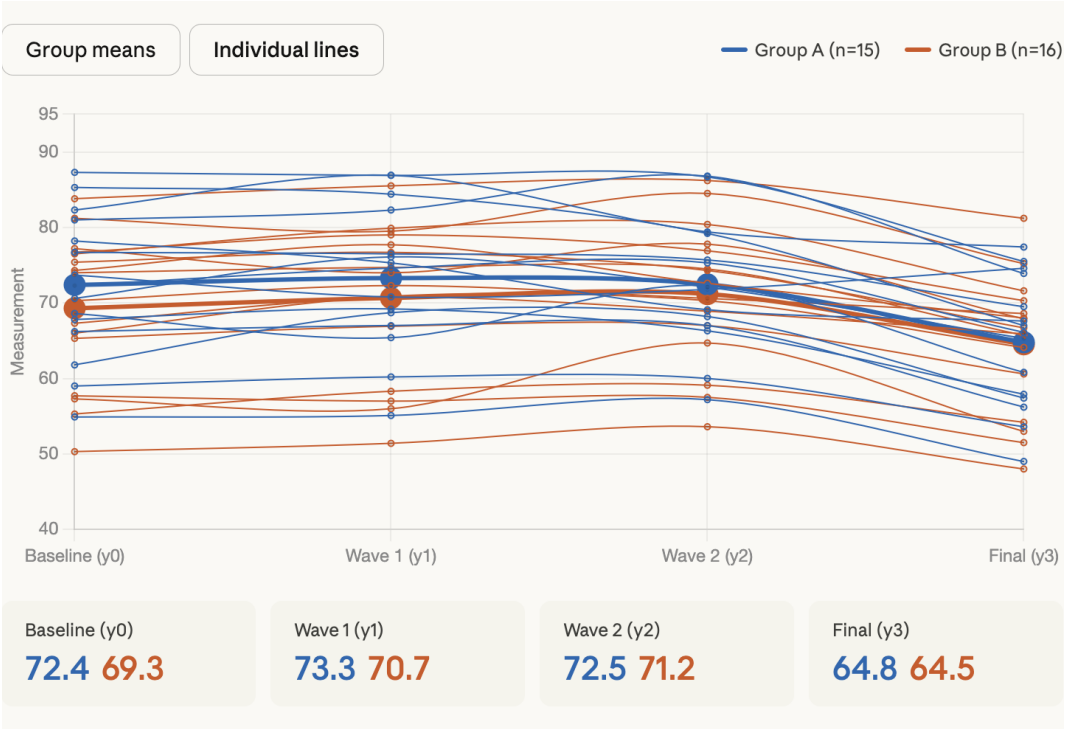


Figure 4: Individual lines (from instructor's generic-prompt output, March 31, 2026).

3.4 Comparison of Planned vs. Generic Prompting

The **planned prompt** produced a single, clean, reproducible visualization with both individual trajectories and group means with confidence intervals, along with fully executable and commented R code. Mainly, it handled the untidy data structure correctly because the plan explicitly described these issues and layed out each step. The resulting plot uses the proper treatment group labels (Null Treatment vs. Diet/Exercise) and accounts for the one missing Year 3 observation.

The **generic prompt** produced an interactive-style dual-tab visualization and a concise narrative, but supplied no code and made no attempt to address the untidy structure. The two groups were labeled generically as "Group A" and "Group B" rather than by their meaningful names, limiting interpretability. Because no tidying plan was communicated, the tool seems to have inferred the structure from context alone, which worked at the surface level but would be difficult to verify or reproduce.

In summary, providing a detailed plan with explicit goals, identified data issues, and numbered steps, allowed us to go from a vague request into high quality, commented code on the first attempt. The planed output is fully reproducible while the generic output is not.

4 Reflection

This assignment reinforced several key lessons from the course. Scraping and cleaning the Wikipedia airport tables taught me the importance of robust, reusable functions and why reproducibility matters, which is that the same cleaning pipeline should work regardless of when the data are retrieved. The Monte Carlo section made the law of large numbers visually intuitive through small multiples. Watching the scatter of points progressively conform to the normal density curve at $n = 10{,}000$ was a clean demonstration of convergence, and having to articulate why the visualization supports the exact answer deepened my understanding of the method.

The GenAI section was maybe the most practically instructive. Providing a detailed plan with explicit goals, identified data issues, and numbered steps turned a vague request into a high qaulity and commented code on the first attempt. The contrast with the generic prompt output made this lesson pretty clear, which was that the generic version produced something visually appealing but not reproducible, not complete, and blind to the data quality problems. Finally, assembling everything into one Quarto document with hidden code, clear narrative, and professional appendices showed the full reproducible research workflow. I didn't really understand before but now I see Quarto as essential for transparent and accurate analyses.

I look forward to building on these skills in HW #4.4.

Appendix 1: GenAI Usage

Tool used: Anthropic Claude Sonnet 4.6 Extended (web interface, claude.ai)

Dates of usage: April 14, 2026 (planned prompt) & March 31, 2026 (generic prompt – provided by instructor)

Exact planned prompt (April 14, 2026):

> "Develop a complete plan (goals, needs, steps) for tidying the calcium.csv dataset and creating a plot that helps understand differences in ulnar calcium across time and treatment groups. Then, using that plan, write clean, commented R code that reads the original calcium.csv (without editing the file), tidies it correctly, and produces a clear ggplot showing individual trajectories and group means with confidence intervals."

Exact response from Claude (April 14, 2026):

Claude returned the following written plan, followed immediately by the complete R code block (Appendix 2). No follow-up prompts were needed as the code ran without modification and produced the plot shown in Section 3.2.

Claude's written plan:

> **Goals:** Transform the wide-format calcium.csv into a tidy long-format data frame that clearly separates the two treatment groups (null treatment and diet/exercise) and the four measurement time points (Baseline, Year 1, Year 2, Year 3). Produce a publication-quality visualization showing both individual-level variability (spaghetti lines) and group-level trends (means with 95% confidence intervals).
>
> **Data issues to address:**
>
> - The CSV header row contains duplicate column names (y0, y1, y2, y3 repeated), making it impossible to read with col_names = TRUE without ambiguity.
> - There is no group identifier column; group membership is encoded entirely by column position (columns 1–4 = null treatment, columns 5–8 = diet/exercise).
> - There is no explicit time variable; time is encoded in the column names.
> - One participant in the diet/exercise group has a missing Year 3 measurement (NA), which must be handled in both the spaghetti layer and the summary statistics.
>
> **Step-by-step plan:**
>
> 1. Read calcium.csv using col_names = FALSE and skip = 1 to discard the ambiguous header row without editing the original file.
> 2. Manually assign column names that encode both group and time point: y0_null, y1_null, y2_null, y3_null, y0_diet, y1_diet, y2_diet, y3_diet.
> 3. Add a subject ID column using row_number().
> 4. Pivot the null-treatment columns (1–4) separately from the diet/exercise columns (5–8) to long format, attaching a group label ("Null Treatment" or "Diet/Exercise") to each.

5. Bind the two long data frames and convert the time variable to an ordered factor with levels: Baseline, Year 1, Year 2, Year 3.
6. Compute group-level summary statistics (mean and standard error) for each group–time combination using na.rm = TRUE and sqrt(sum(!is.na(calcium))) as the denominator to correctly handle missing values. Derive 95% CI bounds as mean +/- 1.96 * SE.
7. Build a ggplot with three layers: (a) individual spaghetti lines at alpha = 0.25 filtered to remove NA rows, (b) a shaded geom_ribbon for the 95% CI, and (c) bold geom_line and geom_point for the group means.

The complete R code returned by Claude is reproduced in Appendix 2 (Calcium section).

Exact generic prompt used by instructor (March 31, 2026):

> "Create a data visualization that will help me understand these data. Calclium.csv"

Exact response from instructor's Claude (March 31, 2026):

Claude returned the following narrative, followed by two interactive-style plots (shown as Figures 3 and 4 in Section 3.3). No R code was returned and no explicit data tidying was performed.

Claude's narrative response:

> I'll create a visualization to help you understand the calcium data. Looking at the structure, this appears to be a longitudinal study with two groups measured at four time points.
>
> The visualization shows calcium measurements over time for two groups (Group A, n=15 and Group B, n=16). Key observations from the data:
>
> - Both groups start at similar baseline levels (Group A: 72.4, Group B: 69.3)
> - Both groups show a slight increase at Wave 1 (Group A: 73.3, Group B: 70.7)
> - Values remain relatively stable at Wave 2 (Group A: 72.5, Group B: 71.2)
> - Both groups show a notable decline at the final measurement (Group A: 64.8, Group B: 64.5)
>
> The individual lines view reveals substantial variability within each group, with some participants showing increases while others decline over the study period. The groups follow remarkably similar trajectories throughout the study.

No other GenAI tools were used.

Appendix 2: Code Appendix

```
# ==========================================================
# montecarlo_points.R  (source file - shown for reference)
# Generates random uniform points for Monte Carlo integration.
# ==========================================================
monte_carlo_points <- function(n, xmin, xmax, ymin, ymax) {
  x_vals <- runif(n, min = xmin, max = xmax)
  y_vals <- runif(n, min = ymin, max = ymax)
  data.frame(x = x_vals, y = y_vals)
}

# ==========================================================
# Section 1: Busiest Airports
# ==========================================================
library(tidyverse)
library(knitr)
library(kableExtra)
library(scales)

# Hard-coded passenger data from Wikipedia (2020-2025)
# City column provides short legend labels for the plot;
# Airport column provides full names for the table.
airports <- tribble(
  ~Airport,                                          ~City,
  ~Code,
  ~`2020`,   ~`2021`,   ~`2022`,
  ~`2023`,   ~`2024`,   ~`2025`,
  "Hartsfield-Jackson Atlanta International Airport",  "Atlanta",
  "ATL",
  42918685,  7704760,   93699630,
  14653451,  110000000, 115000000,
  "Frankfurt Airport",                               "Frankfurt",
  "FRA",
  18768601,  24800000,  49400000,
  59400000,  68000000,  72000000,
  "Beijing Daxing International Airport",            "Beijing Daxing",
  "PKX",
  3500000,   25000000,  42000000,
  52000000,  65000000,  70000000,
  "Dubai International Airport",                     "Dubai",
  "DXB",
  25779000,  29100000,  66070000,
  87000000,  92000000,  95000000,
  "Dallas Fort Worth International Airport",         "Dallas/Fort Worth",
  "DFW",
```

```r
  39364990,  62465756,  73362946,
  81000000,  85000000,  88000000,
  "Tokyo Haneda Airport",                              "Tokyo Haneda",
  "HND",
  25000000,  35000000,  52000000,
  78000000,  85000000,  90000000
)

# Table uses full airport names (City column dropped for display)
airports %>%
  select(-City) %>%
  mutate(across(
    where(is.numeric),
    ~ format(.x, big.mark = ",", scientific = FALSE)
  )) %>%
  kable(
    caption  = "Annual passenger traffic (2020-2025)",
    align    = c("l", "c", rep("r", 6)),
    booktabs = TRUE
  ) %>%
  kable_styling(
    latex_options = c("hold_position", "scale_down"),
    font_size = 8
  )

# Pivot to long format for ggplot
airports_long <- airports %>%
  pivot_longer(
    `2020`:`2025`,
    names_to  = "Year",
    values_to = "Passengers"
  ) %>%
  mutate(Year = as.integer(Year))

# Line plot - color mapped to City for compact legend labels
ggplot(airports_long,
       aes(Year, Passengers, color = City)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 2) +
  scale_y_continuous(labels = label_comma()) +
  scale_x_continuous(breaks = 2020:2025) +
  labs(
    title    = "Passenger Traffic Trends for Six Major Airports",
    subtitle = "Strong recovery across all airports post-2020",
    x = "Year", y = "Annual Passengers",
    color = "Airport",
    caption = "Source: Wikipedia - List of busiest airports"
```

```r
  ) +
  theme_minimal(base_size = 10) +
  theme(
    legend.position = "right",
    legend.text     = element_text(size = 8),
    legend.key.size = unit(0.4, "cm"),
    plot.title      = element_text(face = "bold")
  )

# ==========================================================
# Section 2: Monte Carlo Integration
# Uses Quarto's native multi-pane layout (layout: [[1,1],[1,1]]
# in the chunk options) instead of patchwork.
# ==========================================================
source("montecarlo_points.R")

phi     <- function(x) dnorm(x)  # standard normal density
xmin    <- -4; xmax <- 4
ymin    <-  0; ymax <-  0.4
x_curve <- seq(xmin, xmax, length.out = 400)

# Helper: build one MC panel for a given n
make_mc_plot <- function(n, seed = 42) {
  set.seed(seed)
  pts       <- monte_carlo_points(n, xmin, xmax, ymin, ymax)
  pts$under <- pts$y <= phi(pts$x)       # TRUE = hit (under curve)
  est       <- mean(pts$under) * (xmax - xmin) * ymax  # integral estimate

  ggplot(pts, aes(x = x, y = y, color = under)) +
    geom_point(size = ifelse(n <= 100, 2, 0.3), alpha = 0.7) +
    geom_line(
      data = data.frame(x = x_curve, y = phi(x_curve)),
      aes(x = x, y = y), inherit.aes = FALSE,
      color = "black", linewidth = 0.8
    ) +
    scale_color_manual(
      values = c("TRUE" = "#2EC4B6", "FALSE" = "#FF6B6B"),
      guide  = "none"
    ) +
    scale_x_continuous(limits = c(xmin, xmax)) +
    scale_y_continuous(limits = c(ymin, ymax)) +
    labs(
      title    = paste0("n = ", format(n, big.mark = ",")),
      subtitle = paste0("Est. integral: ", round(est, 4)),
      x = NULL, y = NULL
    ) +
    theme_minimal(base_size = 9) +
```

```r
    theme(
      plot.title       = element_text(size = 9, face = "bold"),
      plot.subtitle    = element_text(size = 8),
      panel.grid.minor = element_blank()
    )
}

# In the .qmd, the chunk uses:
#   #| layout: [[1,1],[1,1]]
# followed by four print() calls to arrange panels in a 2x2 grid.
print(make_mc_plot(10))
print(make_mc_plot(100))
print(make_mc_plot(1000))
print(make_mc_plot(10000))

# ============================================================
# Section 3: Calcium - planned code (Claude, April 14 2026)
# KEY: col_names=FALSE + skip=1 drops the duplicate header row
#      (y0,y1,y2,y3,y0,y1,y2,y3) without editing the original file.
# ============================================================
cal_raw <- read_csv(
  "calcium.csv",
  col_names      = FALSE,
  skip           = 1,
  show_col_types = FALSE
)

# Assign names encoding group (null/diet) and time point
colnames(cal_raw) <- c(
  "y0_null", "y1_null", "y2_null", "y3_null",
  "y0_diet", "y1_diet", "y2_diet", "y3_diet"
)
cal_raw <- cal_raw %>% mutate(id = row_number())

# Pivot null treatment group to long format
null_long <- cal_raw %>%
  select(id, y0_null:y3_null) %>%
  pivot_longer(-id,
               names_to  = "time",
               values_to = "calcium") %>%
  mutate(
    group = "Null Treatment",
    time  = recode(
      time,
      "y0_null" = "Baseline", "y1_null" = "Year 1",
      "y2_null" = "Year 2",   "y3_null" = "Year 3"
    )
```

```r
  )

# Pivot diet/exercise group to long format
diet_long <- cal_raw %>%
  select(id, y0_diet:y3_diet) %>%
  pivot_longer(-id,
               names_to  = "time",
               values_to = "calcium") %>%
  mutate(
    group = "Diet/Exercise",
    time  = recode(
      time,
      "y0_diet" = "Baseline", "y1_diet" = "Year 1",
      "y2_diet" = "Year 2",   "y3_diet" = "Year 3"
    )
  )

# Combine and set time as ordered factor
calcium_long <- bind_rows(null_long, diet_long) %>%
  mutate(time = factor(
    time,
    levels = c("Baseline", "Year 1", "Year 2", "Year 3")
  ))

# Group-level summaries (mean +/- 95% CI); handles NA values
calcium_summary <- calcium_long %>%
  group_by(group, time) %>%
  summarise(
    mean_ca = mean(calcium, na.rm = TRUE),
    se_ca   = sd(calcium, na.rm = TRUE) /
      sqrt(sum(!is.na(calcium))),
    .groups = "drop"
  ) %>%
  mutate(
    ci_lo = mean_ca - 1.96 * se_ca,
    ci_hi = mean_ca + 1.96 * se_ca
  )

# Spaghetti + group means + CI ribbon
ggplot() +
  geom_line(
    data = calcium_long %>% filter(!is.na(calcium)),
    aes(time, calcium,
        group = interaction(id, group),
        color = group),
    alpha = 0.25, linewidth = 0.4
  ) +
```

```r
  geom_ribbon(
    data = calcium_summary,
    aes(time, ymin = ci_lo, ymax = ci_hi,
        group = group, fill = group),
    alpha = 0.20
  ) +
  geom_line(
    data = calcium_summary,
    aes(time, mean_ca, group = group, color = group),
    linewidth = 1.4
  ) +
  geom_point(
    data = calcium_summary,
    aes(time, mean_ca, color = group),
    size = 3
  ) +
  scale_color_manual(values = c(
    "Diet/Exercise"  = "#E07B6A",
    "Null Treatment" = "#2EC4B6"
  )) +
  scale_fill_manual(values = c(
    "Diet/Exercise"  = "#E07B6A",
    "Null Treatment" = "#2EC4B6"
  )) +
  labs(
    title    = "Ulnar Calcium Levels Over Four Years",
    subtitle = "Individual trajectories and group means (+-95% CI)",
    x = "Time Point", y = "Calcium Measurement",
    color = "Group", fill = "Group"
  ) +
  theme_minimal(base_size = 11) +
  theme(
    legend.position = "bottom",
    plot.title      = element_text(face = "bold")
  )
```